

A Comparative Study of SAT Encodings and Exact Solvers for the N-Queens Problem

Vu Minh Hoang^{a,*}

^aVNU University of Engineering and Technology, Hanoi, Vietnam

ARTICLE INFO

Keywords:

N-Queens Problem
SAT Encoding
At-Most-One Constraint
Combinatorial Optimization
Integer Linear Programming
Constraint Programming
CPLEX
Gurobi
CP-SAT

ABSTRACT

This paper compares five at-most-one (AMO) encodings for Boolean satisfiability formulations of empty-board N-Queens: pairwise, binary, commander, sequential counter, and product. The same placement constraints are modeled with integer linear programming (CBC, Gurobi, and CPLEX MIP) and constraint programming (OR-Tools CP-SAT and CPLEX CP Optimizer). An archived benchmark covers $N = 4$ to $N = 200$ with unequal solver coverage and incomplete provenance. Two clean-source runs evaluate all five SAT encodings, CBC, Gurobi, and CP-SAT at $N \in \{20, 50, 100, 150, 200\}$, with three repetitions, one worker, and a 120-s process deadline per run. At $N = 200$, the observed median construction-plus-solver-call times are 2.252 s for Commander, 2.806 s for Product, 3.208 s for Binary, 0.421 s for Gurobi, and 44.776 s for CP-SAT; Sequential and CBC time out in all three repetitions. These times exclude solution extraction and process startup, which are recorded separately. Exact formula-size checks and unit-propagation tests confirm that each implemented AMO encoding enforces the intended constraint and propagates a selected input to the falsity of all others. The datasets are kept separate, and the results do not imply a general ranking beyond the evaluated empty-board instances.

1. Introduction

The N-Queens problem asks for N non-attacking queens on an $N \times N$ board. A valid placement exists for $N = 1$ and every $N \geq 4$, and explicit constructions find one in polynomial time. Nevertheless, the problem provides a structured comparison of modeling and implementation overhead across exact solvers. Its easy decision version should not be confused with the harder N-Queens Completion variant [3, 6].

N-Queens is commonly used to illustrate constraint satisfaction and compare formulations. Results on an empty board do not, by themselves, predict performance on arbitrary scheduling, routing, or other combinatorial problems [6].

The primary motivation of this study is to systematically evaluate how different problem formulations impact the performance of modern automated reasoning tools. Boolean Satisfiability (SAT) solvers have made tremendous strides in the past two decades, largely driven by Conflict-Driven Clause Learning (CDCL) architectures [12]. However, compiling high-level combinatorial constraints into Conjunctive Normal Form (CNF) requires careful consideration. The choice of encoding, particularly for cardinality constraints like “At-Most-One” (AMO), can drastically alter the size of the logical formula and the solver’s ability to prune the search space via unit propagation [17].

The contributions are (i) a common implementation and exact size analysis of five established AMO encodings, (ii) an archived ILP/CP comparison and a separately recorded, common-protocol large-board comparison, and (iii) an explicit account of coverage, timing, and license limitations. No new SAT encoding or general-purpose optimization algorithm is claimed.

Section 2 reviews the background. Section 3 specifies the formulations and encodings. Section 4 reports the archived and controlled measurements with their limitations. Section 5 concludes.

2. Background & Related Work

2.1. The N-Queens Problem

A solution places exactly one queen in each row and column and at most one on each diagonal. The empty-board decision problem has constructive solutions for $N = 1$ and $N \geq 4$; $N = 2, 3$ are infeasible. In contrast,

*Corresponding author

✉ 24021494@vnu.edu.vn (V.M. Hoang)

the decision and counting versions of N-Queens Completion, where some queens are pre-placed, are NP-complete and #P-complete, respectively [6]. The experiments below concern one feasible placement on an empty board, not completion or counting.

2.2. Boolean Satisfiability (SAT)

The Boolean Satisfiability problem (SAT) asks whether there exists an assignment of truth values to a set of boolean variables that makes a given logical formula true. Most solvers require the formula to be in Conjunctive Normal Form (CNF), a conjunction (AND) of clauses, where each clause is a disjunction (OR) of literals.

Modern SAT solving relies heavily on variants of the DPLL (Davis-Putnam-Logemann-Loveland) algorithm augmented with CDCL [12, 13, 15]. Glucose employs clause learning and evaluates learned clauses using literal block distance (LBD), the number of distinct decision levels appearing in a learned clause [1, 2]. The present benchmark uses Glucose4 through PySAT.

2.3. Cardinality Constraints

Many combinatorial problems are expressed using cardinality constraints, which restrict the number of variables in a set $X = \{x_1, x_2, \dots, x_n\}$ that can be true simultaneously. The most critical constraints are:

- **At-Least-One (ALO):** $\sum x_i \geq 1$. In CNF, this is trivially encoded as a single clause: $(x_1 \vee x_2 \vee \dots \vee x_n)$.
- **At-Most-One (AMO):** $\sum x_i \leq 1$. This prevents more than one variable from being true and is the primary bottleneck in translating CSPs to SAT [5].
- **Exactly-One (EXO):** $\sum x_i = 1$. This is logically equivalent to $\text{ALO} \wedge \text{AMO}$.

2.4. Integer Linear Programming (ILP)

ILP is a mathematical optimization framework where the objective function and constraints are linear, and decision variables are restricted to integer values [16]. Solvers typically utilize LP relaxations combined with Branch and Bound and Branch and Cut techniques. Leading commercial ILP solvers include Gurobi [8] and IBM ILOG CPLEX [9], alongside open-source alternatives like CBC via the PuLP library [14].

2.5. Constraint Programming (CP)

Constraint Programming focuses on stating constraints over variables and finding solutions satisfying them, typically using systematic search combined with constraint propagation. A key strength of CP is the availability of “global constraints” like `AllDifferent`, which encapsulate complex subproblems and utilize specialized filtering algorithms (such as Régin’s maximum bipartite matching algorithm [18]) to heavily prune domains [19]. The evaluated engines are Google’s hybrid CP-SAT solver [7] and IBM ILOG CPLEX CP Optimizer [9]. Their internal propagators and search policies should not be inferred from the high-level model alone.

2.6. Related Work

The translation of CSPs to SAT has been extensively studied. Prestwich [17] surveyed various encoding methods. The naive pairwise AMO encoding, while simple, creates $O(n^2)$ clauses, which becomes prohibitive. Sinz [20] introduced the sequential counter encoding, achieving arc consistency with $O(n)$ variables and clauses. Klieber and Kwon [11] proposed the Commander encoding, grouping variables hierarchically. Chen [4] introduced grid and product-based encodings. The five encodings compared here are established methods. The contribution is an empirical comparison for this implementation and dataset rather than a claim of a new encoding.

3. SAT Encodings for N-Queens

3.1. Problem Formulation

Let N be the dimension of the board. A Boolean variable $x_{i,j}$ is defined for each row i and column j ($i, j \in \{1, \dots, N\}$). The variable $x_{i,j}$ is TRUE if a queen is placed at row i , column j , and FALSE otherwise.

The board-level constraint has six components. For each encoding E , auxiliary variables Y are allocated separately for each AMO set, and the resulting CNF $\Phi_E(X, Y)$ satisfies $\Phi_{\text{board}}(X) \iff \exists Y \Phi_E(X, Y)$:

$$\Phi = \Phi_{\text{ALO}}^{\text{row}} \wedge \Phi_{\text{AMO}}^{\text{row}} \wedge \Phi_{\text{ALO}}^{\text{col}} \wedge \Phi_{\text{AMO}}^{\text{col}} \wedge \Phi_{\text{AMO}}^{\text{diag}} \wedge \Phi_{\text{AMO}}^{\text{anti}} \quad (1)$$

3.2. Row and Column Constraints

Every row must contain exactly one queen (EXO constraint), meaning at least one (ALO) and at most one (AMO). The ALO constraint for row i is:

$$\bigvee_{j=1}^N x_{i,j} \quad (2)$$

The AMO constraint depends on the chosen encoding scheme. The exact same logic applies to columns. Thus, for rows and columns, the constraint is $\text{EXO} = \text{ALO} \wedge \text{AMO}$.

3.3. Diagonal Constraints

Queens cannot share diagonals. For main diagonals, the index is defined as $d = i - j$, where $d \in \{-(N-1), \dots, N-1\}$. For anti-diagonals, the index is defined as $s = i + j$, where $s \in \{2, \dots, 2N\}$. Because a diagonal can be empty, only the At-Most-One (AMO) constraint must be enforced for all cells sharing the same d and s . No ALO constraint is required.

3.4. AMO Encoding Schemes

Given a set of n Boolean variables $X = \{x_1, \dots, x_n\}$, the At-Most-One (AMO) constraint specifies that at most one variable in X can evaluate to TRUE:

$$\sum_{i=1}^n x_i \leq 1 \quad (3)$$

Translating this pseudo-Boolean cardinality constraint into Conjunctive Normal Form (CNF) requires balancing formula size against the pruning strength of unit propagation. Five distinct encoding paradigms are implemented and benchmarked in this work.

3.4.1. Binomial (Pairwise) Encoding

The classical direct approach forbids every distinct pair of variables from being simultaneously true:

$$\Phi_{\text{binomial}} = \bigwedge_{1 \leq i < j \leq n} (\neg x_i \vee \neg x_j) \quad (4)$$

This formulation introduces zero auxiliary variables but generates $\binom{n}{2} = O(n^2)$ binary clauses per constraint. When applied across all rows, columns, and diagonals of an $N \times N$ chessboard, the total clause count scales as $O(N^3)$. Notably, whenever any variable x_i is assigned TRUE, standard unit propagation immediately forces all other literals $\neg x_j$ ($j \neq i$), thus achieving full Generalized Arc Consistency (GAC) without search.

3.4.2. Binary Encoding

The Binary encoding compacts the clause footprint by introducing $m = \lceil \log_2 n \rceil$ auxiliary Boolean variables $Y = \{y_0, y_1, \dots, y_{m-1}\}$, which function as a shared binary bit-register [5]. Each index $k \in \{0, \dots, n-1\}$ possesses a unique m -bit binary expansion $k = \sum_{b=0}^{m-1} \text{bit}(k, b) \cdot 2^b$.

The bit-literal mapping function $\ell(k, b)$ is defined as:

$$\ell(k, b) = \begin{cases} y_b & \text{if bit } b \text{ of } k \text{ is 1,} \\ \neg y_b & \text{if bit } b \text{ of } k \text{ is 0.} \end{cases} \quad (5)$$

The complete Binary AMO formula is given in closed form by:

$$\Phi_{\text{binary}} = \bigwedge_{k=0}^{n-1} \bigwedge_{b=0}^{m-1} (\neg x_k \vee \ell(k, b)) \quad (6)$$

If x_k is true, unit propagation fixes every bit in Y . For every $j \neq k$, at least one required bit differs, and the corresponding binary clause immediately propagates $\neg x_j$. Thus this implementation is generalized arc consistent (GAC) with respect to the original AMO inputs; no claim is made about propagation completeness over arbitrary auxiliary assignments. It uses $\lceil \log_2 n \rceil$ auxiliary variables and $n \lceil \log_2 n \rceil$ binary clauses per constraint.

3.4.3. Commander Encoding

Proposed by Klieber and Kwon [11], the Commander encoding decomposes the set X recursively. The variables are partitioned into disjoint groups $G_1, \dots, G_m$ of at most $g = 3$ elements, following the implementation. For each group G_k , a new commander variable c_k is instantiated to act as a group representative.

The encoding consists of four complementary sets of clauses:

1. **Intra-group Pairwise AMO (Φ_{intra}):** Ensures no two variables within the same group can be simultaneously true:

$$\Phi_{\text{intra}} = \bigwedge_{k=1}^m \bigwedge_{\substack{x_i, x_j \in G_k \\ i < j}} (\neg x_i \vee \neg x_j) \quad (7)$$

2. **Forward Implication (Φ_{fwd}):** Dictates that if any element in G_k is true, commander c_k must be true:

$$\Phi_{\text{fwd}} = \bigwedge_{k=1}^m \bigwedge_{x \in G_k} (\neg x \vee c_k) \quad (8)$$

3. **Reverse Implication (Φ_{rev}):** Dictates that commander c_k cannot be true unless at least one variable in G_k is true:

$$\Phi_{\text{rev}} = \bigwedge_{k=1}^m \left(\neg c_k \vee \bigvee_{x \in G_k} x \right) \quad (9)$$

4. **Commander AMO (Φ_{cmd}):** Applies AMO recursively across the set of commander variables:

$$\Phi_{\text{cmd}} = \text{AMO}(\{c_1, c_2, \dots, c_m\}) \quad (10)$$

The recursion terminates at $n \leq 3$ with pairwise AMO. The implementation includes reverse implications, so $c_k \leftrightarrow \bigvee_{x \in G_k} x$. The reverse clauses are optional for the projected AMO relation and for GAC on its input variables; forward implications, intra-group AMO, and recursive commander AMO already propagate $\neg x_j$ after one input x_i becomes true [11]. The implemented version uses $O(n)$ auxiliary variables and clauses.

3.4.4. Sequential Counter Encoding

Introduced by Sinz [20], this encoding constructs an accumulator network. It introduces $n - 1$ auxiliary variables $s_1, \dots, s_{n-1}$, where s_i carries one-way prefix implications needed to detect a second true input; the clauses do not require s_i to equal the prefix disjunction under every satisfying auxiliary assignment.

The clauses enforce prefix propagation and collision prohibition:

$$(\neg x_1 \vee s_1) \quad (11)$$

$$\bigwedge_{i=2}^{n-1} ((\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1})) \quad (12)$$

$$(\neg x_n \vee \neg s_{n-1}) \quad (13)$$

Sequential Counter introduces $n - 1 = O(n)$ auxiliary variables and $3n - 4 = O(n)$ clauses per constraint. It maintains Generalized Arc Consistency via unit propagation: if $x_i = \text{TRUE}$, unit propagation cascades through $s_i \dots s_{n-1}$ to forbid all subsequent x_j ($j > i$), while propagating backward to forbid all antecedent x_j ($j < i$).

3.4.5. Product Encoding

The Product encoding decomposes n variables onto a 2-dimensional grid [4]. Grid dimensions $p = \lceil \sqrt{n} \rceil$ and $q = \lceil n/p \rceil$ are selected such that $n \leq p \times q$. Two sets of auxiliary coordinate variables are introduced: row coordinates $U = \{u_1, \dots, u_p\}$ and column coordinates $V = \{v_1, \dots, v_q\}$.

Each variable x_k ($k \in \{0, \dots, n-1\}$) is mapped to grid coordinates $r = \lfloor k/q \rfloor + 1$ and $c = (k \bmod q) + 1$ via channeling clauses:

$$\Phi_{\text{chan}} = \bigwedge_{k=0}^{n-1} ((\neg x_k \vee u_{\lfloor k/q \rfloor + 1}) \wedge (\neg x_k \vee v_{(k \bmod q) + 1})) \quad (14)$$

Pairwise AMO constraints are then enforced independently across U and V :

$$\Phi_{\text{grid}} = \bigwedge_{1 \leq i < j \leq p} (\neg u_i \vee \neg u_j) \wedge \bigwedge_{1 \leq i < j \leq q} (\neg v_i \vee \neg v_j) \quad (15)$$

To eliminate invalid phantom cells when $p \times q > n$, phantom clauses forbid vacant coordinate pairs:

$$\Phi_{\text{phantom}} = \bigwedge_{r=1}^p \bigwedge_{\substack{c=1 \\ (r-1)q + (c-1) \geq n}}^q (\neg u_r \vee \neg v_c) \quad (16)$$

Threshold Optimization: For short constraint sets ($n < 6$), especially diagonal slices near the corners of the chessboard, generating the 2D grid overhead is counterproductive. The implementation dynamically applies direct pairwise AMO for $n < 6$ and activates the 2D product grid only when $n \geq 6$. If x_k is true, its two coordinate variables become true. Coordinate AMO then falsifies every other coordinate, and each other input differs on at least one coordinate; unit propagation therefore falsifies every other input. The implementation is GAC on original AMO inputs and uses $O(\sqrt{n})$ auxiliary variables and $O(n)$ clauses per constraint.

3.5. Baseline Exact Formulations

3.5.1. Constraint Programming Formulation (CP)

In Constraint Programming, the N-Queens problem is represented compactly using N integer variables $Q = \{q_1, q_2, \dots, q_N\}$, where each variable $q_i \in \{1, \dots, N\}$ designates the column position of the queen in row i . The row constraints are implicitly satisfied by the variable index structure. The column and diagonal constraints are formulated via three global AllDifferent constraints:

$$\text{AllDifferent}(q_1, q_2, \dots, q_N) \quad (17)$$

$$\text{AllDifferent}(q_1 + 1, q_2 + 2, \dots, q_N + N) \quad (18)$$

$$\text{AllDifferent}(q_1 - 1, q_2 - 2, \dots, q_N - N) \quad (19)$$

The model supplies three AllDifferent constraints. Régin's matching algorithm is one possible generalized arc-consistency method [18]; the actual propagation depends on the solver, its version, and presolve settings. In particular, CP-SAT may combine Boolean decompositions, bounds propagation, and matching-based filtering [7]. No filtering complexity for a specific run is inferred from this model description.

3.5.2. Integer Linear Programming Formulation (ILP)

In Integer Linear Programming, the board is modeled using N^2 binary decision variables $x_{i,j} \in \{0, 1\}$, where $x_{i,j} = 1$ if and only if a queen occupies cell (i, j) . The non-attacking requirements are encoded as four systems of linear equations and inequalities:

$$\sum_{j=1}^N x_{i,j} = 1 \quad \forall i \in \{1, \dots, N\} \quad (\text{Row constraints}) \quad (20)$$

$$\sum_{i=1}^N x_{i,j} = 1 \quad \forall j \in \{1, \dots, N\} \quad (\text{Column constraints}) \quad (21)$$

$$\sum_{\substack{1 \leq i, j \leq N \\ i-j=d}} x_{i,j} \leq 1 \quad \forall d \in \{-(N-1), \dots, N-1\} \quad (\text{Main diagonals}) \quad (22)$$

$$\sum_{\substack{1 \leq i, j \leq N \\ i+j=s}} x_{i,j} \leq 1 \quad \forall s \in \{2, \dots, 2N\} \quad (\text{Anti-diagonals}) \quad (23)$$

Table 1

Asymptotic Complexity of N-Queens SAT Encodings

Encoding Scheme	Total Variables	Total Clauses	Input GAC via UP
Binomial (Pairwise)	N^2	$O(N^3)$	Yes
Binary	$N^2 + O(N \log N)$	$O(N^2 \log N)$	Yes
Commander	$N^2 + O(N^2)$	$O(N^2)$	Yes
Sequential Counter	$N^2 + O(N^2)$	$O(N^2)$	Yes
Product	$N^2 + O(N\sqrt{N})$	$O(N^2)$	Yes

3.6. Complexity Comparison

Table 1 summarizes the asymptotic complexities of the implemented AMO encodings when applied to the full N-Queens problem. For an $N \times N$ board, the AMO constraint is applied to N rows, N columns, and $O(N)$ diagonals (each of length up to N).

Here, input GAC means that setting one input of an individual AMO constraint to true makes unit propagation falsify its other inputs. It does not imply complete propagation for the full N-Queens formula. For $N \geq 2$, let $a_E(k)$ and $c_E(k)$ count auxiliary variables and clauses for one k -input AMO, with both zero for $k \leq 1$. Exact board-level counts are

$$V_E(N) = N^2 + (2N + 2)a_E(N) + 4 \sum_{k=2}^{N-1} a_E(k), \quad (24)$$

$$C_E(N) = 2N + (2N + 2)c_E(N) + 4 \sum_{k=2}^{N-1} c_E(k). \quad (25)$$

The $2N$ term counts row and column ALO clauses. The one-constraint counts for pairwise, Binary, and Sequential are

$$(a_{\text{pair}}, c_{\text{pair}}) = (0, \binom{k}{2}), \quad (26)$$

$$(a_{\text{binary}}, c_{\text{binary}}) = (\lceil \log_2 k \rceil, k \lceil \log_2 k \rceil), \quad (27)$$

$$(a_{\text{seq}}, c_{\text{seq}}) = (k - 1, 3k - 4). \quad (28)$$

Product uses pairwise AMO for $k < 6$. For larger k , let $p = \lceil \sqrt{k} \rceil$ and $q = \lceil k/p \rceil$. Its counts are

$$(a, c) = \left(p + q, 2k + \binom{p}{2} + \binom{q}{2} + pq - k \right). \quad (29)$$

For Commander, groups have sizes $t_j \leq 3$ and $m = \lceil k/3 \rceil$. When $k > 3$, the recursive counts are

$$a(k) = m + a(m), \quad c(k) = \sum_j \binom{t_j}{2} + k + m + c(m). \quad (30)$$

The pairwise case is the recursion base. The m reverse clauses are included in the count. These expressions count generated clauses before solver preprocessing.

4. Experimental Results & Comparative Analysis

4.1. Experimental Setup

The archived dataset contains 116 aggregate rows across $N \in \{4, 8, 10, 15, 20, 25, 30, 40, 50, 75, 100, 150, 200\}$. It was collected on a Windows machine described in the original experiment as having an Intel Core processor and 16 GB of RAM; the exact CPU model, package versions at collection time, random seeds, solver thread counts, and per-run measurements were not retained. These omissions limit independent reproduction and statistical inference. The five SAT formulations use Glucose4 through PySAT [10]; the comparison models use CBC through PuLP, Gurobi, CPLEX MIP, OR-Tools CP-SAT, and CPLEX CP Optimizer [7–9, 14].

Table 2

Variables and Clauses generated by each SAT Encoding

Encoding	$N = 20$		$N = 50$		$N = 100$	
	Vars	Clauses	Vars	Clauses	Vars	Clauses
Binomial	400	12,580	2,500	203,450	10,000	1,646,900
Binary	866	7,296	4,036	57,244	13,678	269,024
Commander	1,172	5,050	7,514	33,740	30,536	138,628
Sequential	1,882	4,372	12,202	28,912	49,402	117,812
Product	1,190	4,524	5,890	30,330	19,428	117,736

Table 3

Archived construction plus solver invocation time (s); solution extraction excluded

N	Binomial	Binary	Commander	Sequential	Product
8	0.0018	0.0022	0.0021	0.0018	0.0025
20	0.0235	0.3306	0.0192	0.0324	0.6719
50	0.3625	0.1363	0.1212	3.1294	34.7981
75	1.2642	0.3998	0.8540	6.4575	—
100	2.8812	0.5815	0.8140	68.1819	—
150	—	1.5867	—	—	—
200	—	3.2073	—	—	—

The archived CSV reports construction time, solver invocation time, and their sum in seconds. SAT solution extraction was performed after the solver timer stopped; equivalent postprocessing was also outside the reported total for the other models. Therefore, the displayed times are *not* end-to-end latencies. The historical scripts contain both a 120-s, two-repeat configuration and a 300-s, three-repeat configuration, and raw repetitions are unavailable. The exact originating protocol for each archived row cannot be recovered. The revised runner records per-repeat status, full-process deadlines, wall time, validation, package versions, and worker count without overwriting this archive; its outputs are kept separate from the historical tables and figures below.

Solver coverage differs. Only Binary among the five SAT encodings was recorded at $N = 150$ and $N = 200$; Product ends at $N = 50$, while pairwise, Commander, and Sequential end at $N = 100$. CBC ends at $N = 100$. Gurobi and CP-SAT have recorded results through $N = 200$. Missing entries must be read as *not evaluated*, except entries marked with a license symbol. CPLEX MIP Community is limited to 1,000 variables or constraints; its $N = 40$ model already has 1,600 binary variables. CP Optimizer Community applies a search-space limit of 2^{1000} , which prevents the recorded $N = 150$ and $N = 200$ models from being evaluated under that license. These limits differ from the academic Gurobi license used for the archived Gurobi rows.

4.2. SAT Encoding Comparison

Table 2 illustrates the formula size generated by different encodings at key problem sizes.

Table 2 lists generated CNF sizes, while Figure 1 plots the sizes present in the archived CSV. Pairwise AMO has no auxiliary variables but reaches 1,646,900 clauses at $N = 100$. Sequential uses 49,402 total variables at that size. Product uses 117,736 clauses at $N = 100$, slightly fewer than Sequential's 117,812, although no Product runtime is recorded there. The Product size at $N = 100$ is computed from the unchanged encoder and the exact counting recurrence above; it is neither a new timing observation nor a point in Figure 1.

Table 3 reports the archived construction-plus-solver-call time for SAT; dashes mean that a runtime was not recorded. Figure 2 shows construction time, while Figure 3 shows solver invocation time.

Within the common SAT measurements, Commander has the lowest recorded total at $N = 20$ (0.0192 s) and $N = 50$ (0.1212 s), while Binary is lowest at $N = 75$ (0.3998 s) and $N = 100$ (0.5815 s). At $N = 8$, pairwise and Sequential both round to 0.0018 s; the underlying CSV values differ by only six microseconds, so no meaningful winner is identified. Binary's 1.5867 s and 3.2073 s at $N = 150$ and $N = 200$ have no same-size SAT competitor in the archive.

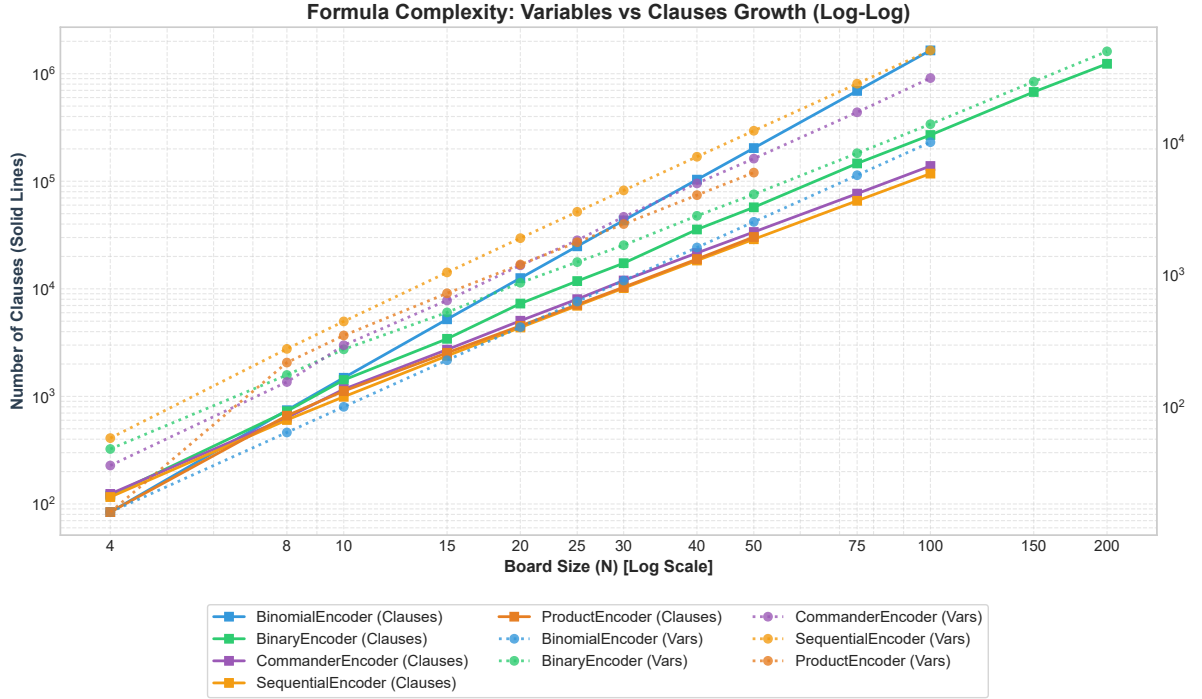


Figure 1: Archived CNF sizes by SAT encoding. Solid lines give clause counts on the left axis, and dotted lines give total variables on the right axis; both axes are logarithmic.

Table 4

Archived construction plus solver invocation time (s): retrospective best SAT and baselines

N	Best SAT	ILP (CBC)	ILP (Gurobi)	ILP (CPLEX)	CP (CPLEX)	CP-SAT	Best Method
10	0.003	0.068	0.012	0.024	0.044	0.016	Best SAT
20	0.019	1.644	0.014	0.033	0.046	0.051	Gurobi
30	0.080	0.229	0.029	0.057	0.044	0.111	Gurobi
40	0.098	0.341	0.075	— [†]	0.048	0.212	CPLEX CP
50	0.121	0.791	0.151	— [†]	0.048	0.366	CPLEX CP
75	0.400	3.202	0.184	— [†]	0.057	0.879	CPLEX CP
100	0.581	10.413	0.150	— [†]	0.097	1.819	CPLEX CP
150	1.587	—	0.364	— [†]	— [†]	14.953	Gurobi
200	3.207	—	0.601	— [†]	— [†]	22.487	Gurobi

[†] CPLEX Community license limit; other dashes mean not evaluated.

Sequential's 68.1819 s at $N = 100$ is an observation; the available data do not establish whether auxiliary-variable branching, propagation cost, or clause learning caused it.

4.3. SAT vs ILP vs CP Comparison

Table 4 reports the minimum recorded SAT time for each N alongside the ILP and CP baselines. This “Best SAT” column is a retrospective lower envelope, not a solver-selection procedure available before running the encodings.

Table 4, Figure 4, and Figure 5 show the recorded comparison through $N = 200$. The best-SAT series is a retrospective lower envelope; at $N = 150$ and $N = 200$ it contains only Binary/Glucose4, because no other SAT encoding was measured there. Gurobi has the lowest listed construction-plus-solver-call time at $N = 150$ (0.364 s) and $N = 200$ (0.601 s) under its academic license. At these sizes, Binary/Glucose4 records 1.587 s and 3.207 s, while CP-SAT records 14.953 s and 22.487 s. The $N = 200$ ratio of CP-SAT to Binary is approximately 7.01 for this recorded metric; it does not include solution extraction or establish a general ranking among SAT encodings. CPLEX

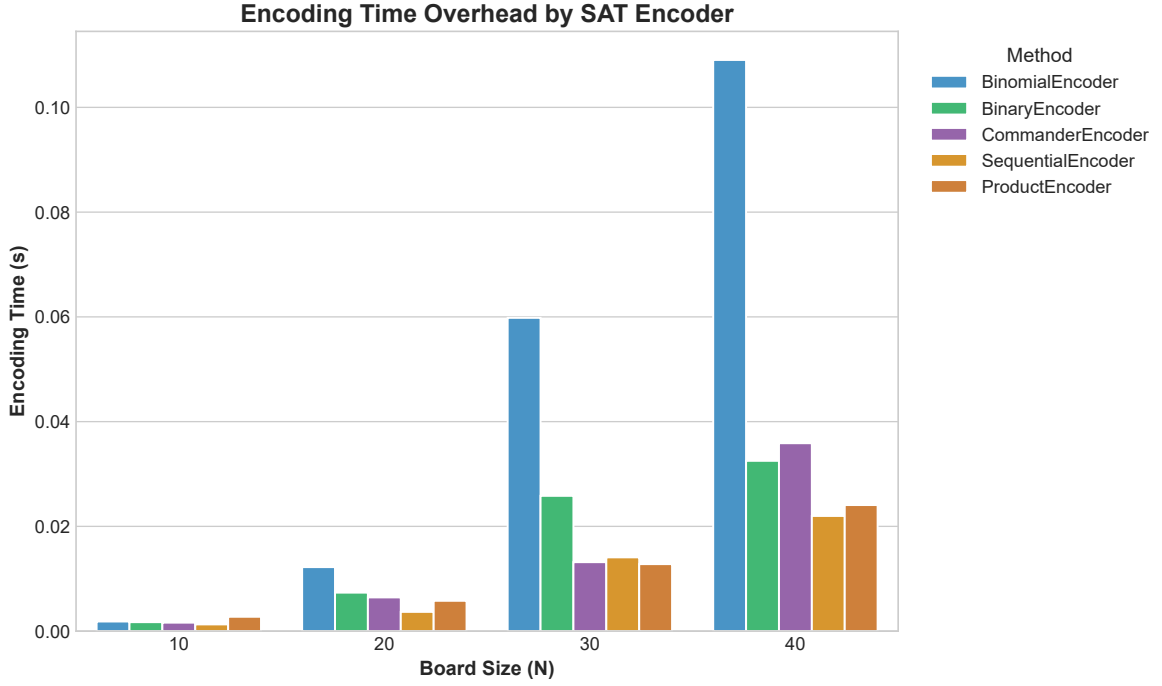


Figure 2: Archived SAT formula-construction time for $N \in \{10, 20, 30, 40\}$; linear time scale.

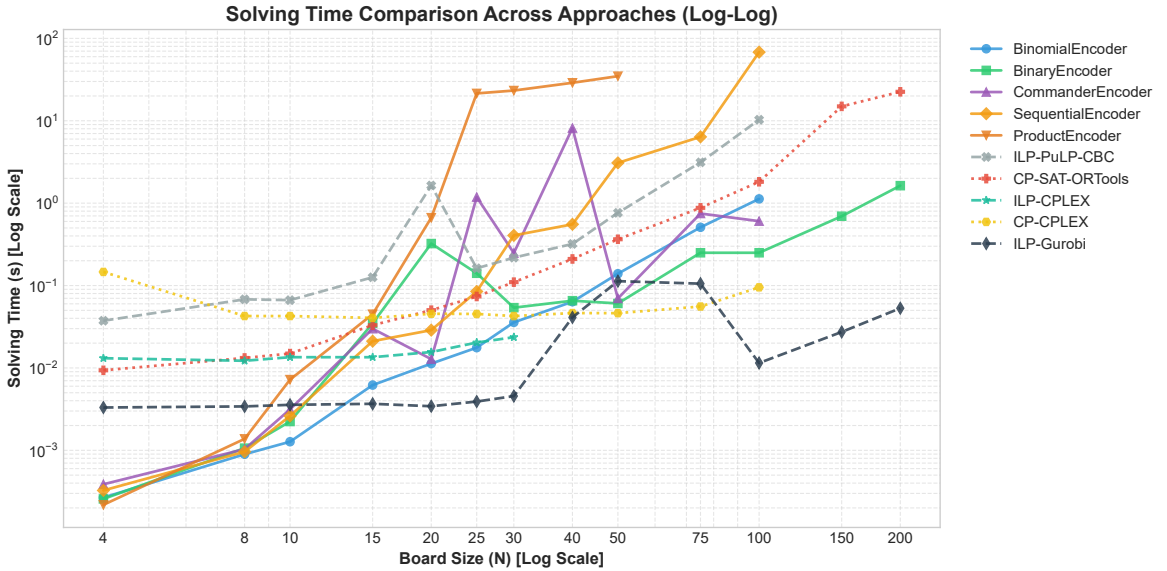


Figure 3: Archived solver invocation time for SAT encodings and ILP/CP baselines on logarithmic axes. Missing method-size combinations were not measured.

Community license limits and deliberately unmeasured cases prevent a complete method-by-size matrix. CBC records 10.413 s at $N = 100$, but a small, non-monotone series does not establish an asymptotic growth law. No solver-internal presolve or clause-learning explanation is inferred from these times.

Figure 6 sorts each method's recorded construction-plus-solver-call times and plots the time of the corresponding case. Its horizontal extent reflects the number of archived measurements, not a common set of attempted cases.

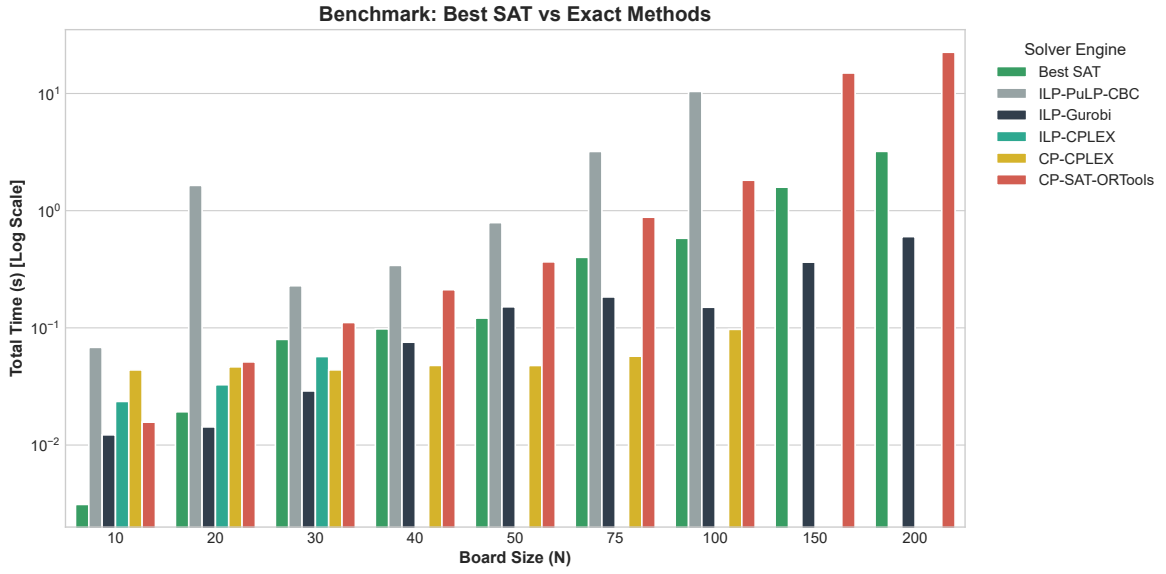


Figure 4: Archived construction plus solver invocation time for the retrospective best SAT encoding at each plotted size and the ILP/CP baselines, $N \in \{10, 20, 30, 40, 50, 75, 100, 150, 200\}$. The best-SAT bars may represent different encodings at different sizes; the vertical axis is logarithmic.

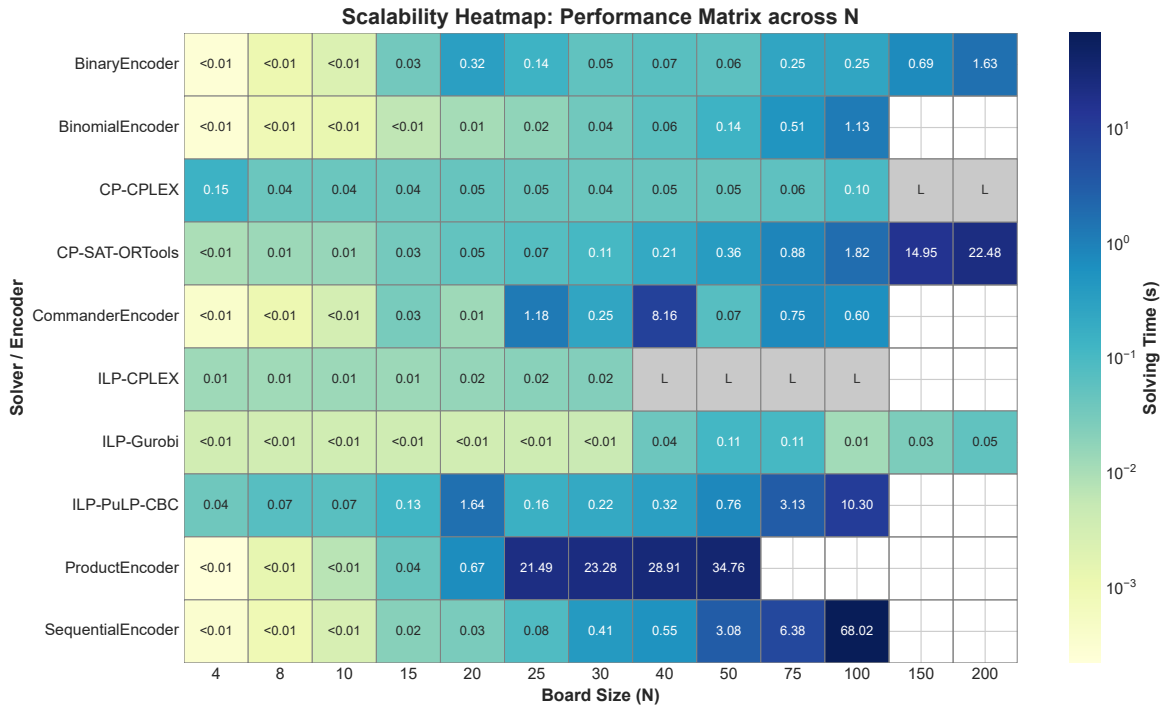


Figure 5: Archived solver invocation time in seconds on a logarithmic color scale. Blank cells were not measured; grey cells marked L denote archived CPLEX Community license failures. Values below 0.01 s are labeled < 0.01.

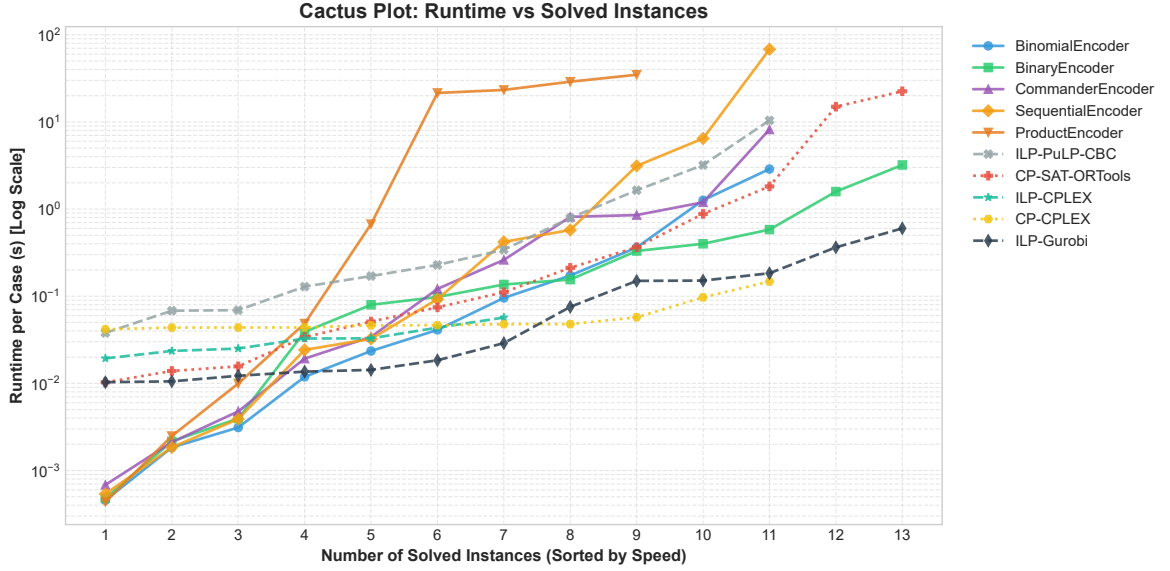


Figure 6: Cactus profile of successful archived construction-plus-solver-call times, sorted separately for each method. The ordinate is per-case time, not cumulative time; a shorter curve reflects fewer recorded cases, not a timeout.

4.4. Controlled Protocol Check

A separate run evaluated Binary/Glucose4, Gurobi, and CP-SAT at five common sizes with three repetitions per method and size, one solver worker, a 120-s full-process deadline, and validation of every returned placement. The machine ran Windows 11 on an AMD processor with 16 logical CPUs. Python 3.14.4, PySAT 1.9.dev15, Gurobi 13.0.3, and OR-Tools 9.15.6755 were recorded in the accompanying metadata. The source working tree was modified at collection time, so this run is not tied to a clean commit. Table 5 gives the median construction-plus-solver-call time; all 45 runs returned valid placements. The raw repetitions, summary, and environment metadata are supplied in the `new_benchmark/` directory under `nqueens_sat/results/benchmarks/`.

Table 5: Separate controlled run: median construction plus solver invocation time (s), three repetitions per cell

N	Binary/Glucose4	Gurobi	CP-SAT
20	0.296	0.034	0.047
50	0.140	0.139	0.325
100	0.509	0.120	2.013
150	1.469	0.276	13.269
200	3.163	0.496	49.921

At $N = 200$, the corresponding median within-worker wall times, including solution extraction, were 3.166, 0.567, and 49.921 s. The controlled CP-SAT median differs substantially from the archived 22.487 s, while the archived run lacks enough configuration detail to isolate the cause. Results from the two datasets are therefore not pooled or interpreted as a statistical comparison. This check confirms that the revised protocol records complete, valid runs; it does not establish a ranking for the SAT encodings absent from this run.

4.5. Clean-Source Common-Size Comparison

Two additional runs applied the same eight methods to $N \in \{20, 50, 100\}$ and $N \in \{150, 200\}$. Each method–size cell was repeated three times with one solver worker and a 120-s deadline covering the complete worker process. Both runs used clean working trees and identical Python source files. Their public metadata record the same SHA-256 source fingerprint. The smaller-board and large-board data are stored in `common_small_benchmark/` and `large_n_benchmark/`, respectively, under `nqueens_sat/results/benchmarks/`. Both directories include raw repetitions, summaries, package versions, and environment metadata. Returned placements were validated. CPLEX was excluded because the available Community licenses prevent a like-for-like comparison over the full size range. These two runs are distinct from the archive and the earlier 45-run protocol check.

Table 6 gives median construction-plus-solver-call times at the three smaller sizes. All five SAT encodings completed every repetition. CBC timed out in all three $N = 20$ attempts despite completing at $N = 50$ and $N = 100$; this non-monotone observation is not evidence of an asymptotic law or proof that the $N = 20$ model is infeasible. The raw CSV retains each run’s status and duration.

Table 6: Clean-source smaller-board run: median construction plus solver invocation time (s), three repetitions per cell

Method	$N = 20$	$N = 50$	$N = 100$
Binomial/Glucose4	0.025	0.367	3.237
Binary/Glucose4	0.300	0.131	0.534
Commander/Glucose4	0.019	0.133	0.708
Sequential/Glucose4	0.036	3.000	71.145
Product/Glucose4	0.581	28.036	36.796
CBC	Timeout (3/3)	0.771	9.805
Gurobi	0.035	0.124	0.132
CP-SAT	0.044	0.311	1.740

Table 7 reports the median and the minimum–maximum range of construction-plus-solver-call time at the two larger sizes for cells with three successful runs. The range describes the observed repetitions; it is not a confidence interval. A timeout has no measured completion time and is excluded from a median.

Table 7: Clean-source large-board run: median [minimum, maximum] time (s) over three repetitions per cell

Method	$N = 150$	$N = 200$
Binomial/Glucose4	10.397 [10.352, 10.798]	24.805 [24.300, 25.338]
Binary/Glucose4	1.519 [1.391, 1.540]	3.208 [3.054, 3.220]
Commander/Glucose4	1.253 [1.251, 1.288]	2.252 [2.222, 2.358]
Sequential/Glucose4	108.028 [103.949, 108.953]	Timeout (3/3)
Product/Glucose4	1.162 [1.112, 1.205]	2.806 [2.262, 2.860]
CBC	62.080 [60.012, 62.214]	Timeout (3/3)
Gurobi	0.294 [0.260, 0.754]	0.421 [0.420, 0.475]
CP-SAT	12.252 [11.933, 12.491]	44.776 [38.479, 45.935]

Product has the lowest observed SAT median at $N = 150$, and Commander has the lowest at $N = 200$. Both differ from the archive’s apparent “Best SAT” at these sizes because Binary was its only measured SAT encoding. Gurobi has the lowest recorded median among the tested paradigms at both sizes. Sequential and CBC reach the full-process deadline in all three $N = 200$ runs; no runtime or SAT/UNSAT conclusion is assigned to those cells. As with the archived metric, the reported times exclude solution extraction and worker startup. Only five empty-board sizes and three repetitions per cell were tested, so neither asymptotic behavior nor performance on completion instances follows from this ranking.

4.6. Propagation and Interpretation of the Measurements

For an isolated AMO constraint, pairwise clauses immediately falsify all other inputs after one input becomes true. The Binary encoding fixes a unique auxiliary bit pattern, and every other input disagrees with at least one bit. The Product encoding fixes one row and one column coordinate and eliminates every other input through coordinate

AMO. Commander and Sequential propagate the same input consequence through their respective hierarchical and prefix structures. Thus all five implemented encodings are GAC on the original inputs of one AMO constraint. This statement does not imply that unit propagation alone solves the complete N-Queens formula.

The structural differences could affect Glucose4's branching, propagation, conflicts, and learned clauses [1, 12]. The archive has no solver-internal counters; the clean-source raw data record aggregate conflicts, decisions, propagations, and restarts for completed SAT runs, but not decisions on auxiliary variables, learned-clause LBD distributions, deletion events, or controlled ablations. In particular, a long sequential implication chain can be traversed within one decision level; its length does not imply high LBD. Likewise, binary original clauses do not guarantee short learned clauses. The observed runtime ordering is therefore reported without attributing it to VSIDS distortion or low-LBD "glue" clauses.

The historical plots have unequal solver coverage and no raw repetition-level uncertainty. The two clean-source runs supply raw repetitions for five common sizes but no independent machine or seeded ablation. The N-Queens empty-board feasibility problem has known polynomial constructions, so trends from this instance family should not be generalized to hard completion instances or other combinatorial domains [6]. A broader follow-up should fix seeds where supported, use additional machines, and include both SAT and UNSAT completion instances.

5. Conclusion & Future Work

This study compared five established SAT AMO encodings and ILP/CP baselines for finding one placement on an empty N-Queens board. Formula sizes follow the stated construction, and the archived tables agree with the CSV to the displayed precision. In the archive, Binary/Glucose4 records 3.207 s at $N = 200$, compared with 22.487 s for CP-SAT and 0.601 s for Gurobi; no other SAT encoding was recorded at that size. The two clean-source runs cover all five SAT encodings at $N \in \{20, 50, 100, 150, 200\}$. At $N = 200$, Commander, Product, and Binary record medians of 2.252, 2.806, and 3.208 s, respectively, while Sequential and CBC time out in all three attempts. These results correct the coverage gap but do not establish a general scalability claim. The construction-plus-solver-call metric excludes solution extraction, and the historical run-level provenance remains incomplete.

Further work should compare Commander with and without reverse implications, test Product threshold and phantom-clause choices, and evaluate a valid symmetry-breaking constraint under the same protocol for every paradigm. Solver-level counts of conflicts, decisions, propagations, and learned-clause LBD would permit the proposed CDCL explanations to be tested. Harder N-Queens Completion or blocked-board families, including infeasible instances, would broaden the evidence beyond empty-board feasibility.

CRedit Authorship Contribution Statement

Vu Minh Hoang: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization.

Declaration of Competing Interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of Generative AI and AI-Assisted Technologies

During the preparation of this work, the author utilized generative AI tools to assist in code optimization, visualization aesthetics, and LaTeX formatting. The author reviewed and edited the manuscript and takes full responsibility for the content.

Data Availability

The source, archived aggregate CSV, figures, and this manuscript are available at <https://github.com/Ohhalo02/nqueens-sat-optimization-01>. The earlier controlled check and two clean-source experiments have per-repeat records, summaries, and metadata in separate directories under `nqueens_sat/results/benchmarks/`. The clean-source metadata include a SHA-256 fingerprint of the Python files used by both runs. The historical CSV has no raw repetitions or complete provenance; the four datasets are not pooled.

References

- [1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI-09), pp. 399–404.
- [2] Audemard, G., Simon, L., 2018. Glucose in the SAT 2018 competition, in: Proceedings of SAT Competition 2018: Solver and Benchmark Descriptions, pp. 18–19. URL: <https://helda.helsinki.fi/handle/10138/237063>.
- [3] Bell, J., Stevens, B., 2009. A survey of known results and research areas for n-queens. Discrete Mathematics 309, 1–31. URL: <https://doi.org/10.1016/j.disc.2007.12.043>.
- [4] Chen, J., 2010. A new SAT encoding of the at-most-one constraint, in: Proceedings of the Tenth International Workshop on Constraint Modelling and Reformulation (ModRef 2010), pp. 1–9. URL: <https://archive.modref.org/files/papers/2010/ModRef2010-11-A-New-SAT-Encoding-of-the-At-Most-One-Constraint.pdf>.
- [5] Frisch, A.M., Giannaros, P.A., 2010. SAT encodings of the at-most- k constraint: Some old, some new, some fast, some slow, in: Proceedings of the Tenth International Workshop on Constraint Modelling and Reformulation (ModRef 2010), pp. 1–15. URL: <https://archive.modref.org/files/papers/2010/ModRef2010-13-SAT-Encodings-of-the-At-Most-k-Constraint.pdf>.
- [6] Gent, I.P., Jefferson, C., Nightingale, P., 2017. Complexity of n -Queens Completion. Journal of Artificial Intelligence Research 59, 815–848. URL: <https://doi.org/10.1613/jair.5512>.
- [7] Google Developers, 2019. Or-tools. URL: <https://developers.google.com/optimization>.
- [8] Gurobi Optimization, LLC, 2025. Gurobi optimizer reference manual. URL: <https://www.gurobi.com>.
- [9] IBM Corporation, 2022. Ibm ilog cplex optimization studio cplex user's manual. URL: <https://www.ibm.com/analytics/cplex-optimizer>.
- [10] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: International Conference on Theory and Applications of Satisfiability Testing, Springer, pp. 428–437. URL: https://doi.org/10.1007/978-3-319-94144-8_26.
- [11] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects. Fourth Workshop on Constraints in Formal Verification (CFV 2007). URL: https://www.cs.cmu.edu/~wklieber/papers/2007_efficient-cnf-encoding-for-selecting-1.pdf.
- [12] Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning SAT solvers, in: Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), Handbook of Satisfiability. IOS Press, volume 185, pp. 131–153. URL: <https://doi.org/10.3233/978-1-58603-929-5-131>.
- [13] Marques-Silva, J.P., Sakallah, K.A., 1999. GRASP: A search algorithm for propositional satisfiability. IEEE Transactions on Computers 48, 506–521. URL: <https://doi.org/10.1109/12.769433>.
- [14] Mitchell, S., O'Sullivan, M., Dunning, I., 2011. PuLP: A Linear Programming Toolkit for Python. Technical Report. University of Auckland. URL: <https://optimization-online.org/wp-content/uploads/2011/09/3178.pdf>.
- [15] Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S., 2001. Chaff: Engineering an efficient SAT solver, in: Proceedings of the 38th Annual Design Automation Conference, pp. 530–535. URL: <https://doi.org/10.1145/378239.379017>.
- [16] Nemhauser, G.L., Wolsey, L.A., 1988. Integer and Combinatorial Optimization. John Wiley & Sons. URL: <https://doi.org/10.1002/9781118627372>.
- [17] Prestwich, S.D., 2009. CNF encodings, in: Handbook of Satisfiability. IOS Press, volume 185, pp. 75–97. URL: <https://doi.org/10.3233/978-1-58603-929-5-75>.
- [18] Régin, J.C., 1994. A filtering algorithm for constraints of difference in cps, in: Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI-94), pp. 362–367.
- [19] Rossi, F., van Beek, P., Walsh, T. (Eds.), 2006. Handbook of Constraint Programming. Elsevier. URL: <https://shop.elsevier.com/books/handbook-of-constraint-programming/rossi/978-0-444-52726-4>.
- [20] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: International Conference on Principles and Practice of Constraint Programming, Springer, pp. 827–831. URL: https://doi.org/10.1007/11564751_73.